동적계획법 (Dynamic Programming)

벨만 최적방정식과 정책평가 · 정책 반복과 GridWorld 실습 · 가치 반복, 수렴 증명, 모델 기반의 전제

Machine Learning 2 · 4주차

한국과학영재학교 (KSA)

Week 4

이번 주차 개요

Week 3에서 MDP를 정식화했지만, “무한히 먼 미래까지 내다보아야 하는 것처럼 보이는” 누적 보상 최대화 문제는 여전히 **계산 불가능**해 보인다. 벨만의 통찰은 이 무한합을 **재귀적 관계**로 다시 쓸 수 있다는 것(3.3절 벨만 기대방정식). 이번 장은 그 재귀식을 **실제로 계산하는 절차**를 다룬다.

이 주차를 마치면

- 벨만 기대방정식으로 주어진 정책의 V^π, Q^π 를 **반복적 정책평가**로 계산한다.
- 정책 반복의 두 단계가 왜 필요하고, **“멈추면 최적”**인지를 정책개선 정리(단조 증가) + 유한성으로 설명한다.
- 가치 반복(평가 한 번 + 바로 개선)을 비교하고, **축소 사상(바나흐 고정점 정리)**이 수렴을 보장함을 설명한다.
- **모델 기반**(P 를 정확히 아는) 전제의 한계를 짚고, model-free(Week 5–6)의 필요성을 예측한다.

세 차시 (5칸 GridWorld 위에서 직관이 쌓인다)

1. **4.1 벨만 최적방정식과 정책평가** – 연립방정식 직접 풀이 vs 반복, V^π/Q^π 다리, 최적방정식 유도.
2. **4.2 정책 반복과 GridWorld 실습** – 평가$\rightarrow$개선을 GridWorld에서 돌리고 “멈추면 최적”을 정리로, 수정된 정책 반복까지.
3. **4.3 가치 반복, 수렴 증명, 모델 기반 전제** – “한 줄 차이”인 가치 반복, k스텝 내다보기, 바나흐 고정점 정리, 전제 한계.

한 줄 감각: 벨만방정식은 **연립방정식이 아니라 반복 대입으로 푸는 것**. 이 감각은 Week 6 TD(경험으로 근사)와 Week 9 DQN(신경망으로 고정점 찾기)까지 그대로 재사용된다.

4.1 벨만 최적방정식과 정책평가

이 절에서 무엇을 만들 것인가

1. **벨만 연산자 T^π**
2. **V^π 와 Q^π 사이의 다리**

4.2·4.3절의 모든 알고리즘은 4.1절의 이 세 가지 (연산자 반복 · Q^π 점수표 · max)의 조합일 뿐이다.

복습: 벨만 기대방정식과 벨만 연산자 T^π

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$

지금 상태의 가치 = 지금 당장 받는 보상 + 할인된 다음 상태들의 가치

$$V^\pi(s) = \sum_a \pi(a|s) \Big[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s') \Big]$$

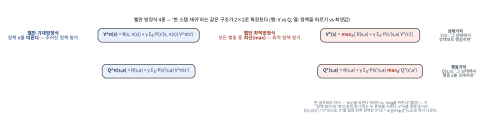
벨만 연산자 T^π

$$V_0 \xrightarrow{T^\pi} V_1 \xrightarrow{T^\pi} \cdots \xrightarrow{T^\pi} V^\pi$$

V^π 와 Q^π: 동전의 양면, 그리고 2×2

$$Q^\pi(s, \pi(s)) = V^\pi(s)$$

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s')$$



네 개의 벨만방정식: **기대 vs 최적** × V **vs** Q 로 2×2.

	기대(π 를 따른다)	**최적**(max)
V	V^π (4.1절 = 오늘)	V^* (4.2–4.3절)
Q	Q^π	Q^* (Week 9 DQN)

이번 절의 모든 코드와 수식이 이 다리(V-방정식 ↔ Q-방정식 사이)를 통해 오간다.

실습 (1/3): 3-state 순환 MDP를 두 길로

"연립 일차방정식"을 직접 만진다. 3.3절 2상태 왕복 예(행동 1개)를 **확률적 전이** 3상태 버전으로 ($\gamma = 0.9$): 상태 0: $+1$, **항상** 1로. 상태 1: $+2$, 2로 0.7 / 0으로 0.3. 상태 2: 0, **항상** 0으로. 행동을 하나뿐이므로 정책은 자명 — **평가할 것: 상태 0·1·2의 가치?**

```
P = {0: {0: [(1.0, 1)]},
     1: {0: [(0.7, 2), (0.3, 0)]},
     2: {0: [(1.0, 0)]}}
R = {0: {0: 1.0}, 1: {0: 2.0}, 2: {0: 0.0}}
policy = [0, 0, 0];  gamma = 0.9
```

방법 1 (연립방정식 직접 풀이): 세 방정식 $V(0) = 1 + 0.9\,V(1)$, $V(1) = 2 + 0.9\,(0.7\,V(2) + 0.3\,V(0))$, $V(2) = 0.9V(0)$ 를 대입·정리하면 $0.2467\,V(0) = 2.8 \Rightarrow V(0) \approx 11.3498$, $V(1) \approx 11.4998$, $V(2) \approx 10.2148$. (검산: $1 + 0.9 \times 11.4998 \approx 11.3498$ ✓. Week 2.2 정규방정식과 같은 가족.)

실습 (2/3): 방법 2 — 반복적 대입(sweep)

같은 값을 **직접 풀지 않고** 반복법으로. V 를 0으로 시작해 0→1→2 순서로 **in-place** 한 스텝씩 갱신:

sweep	$V(0)$	$V(1)$	$V(2)$	최대 변화 Δ
1	1.0000	2.2700	0.9000	2.2700
2	3.0430	3.3886	2.7387	2.0430
3	4.0497	4.8188	3.6448	1.4302
5	6.1635	6.6902	5.5471	0.9530
7	7.6514	8.0469	6.8862	0.6565
⋮	⋮	⋮	⋮	($\approx \times 0.9$씩)
88	**11.3498**	**11.4998**	**10.2148**	$< 10^{-6} \rightarrow$ 종료

확인할 것

88회 sweep을 거치고 나서야 $\Delta < 10^{-6}$ — 방법 1과 **같은 값**$(11.3498, 11.4998, 10.2148)$. 두 방법은 "**같은 고정점을 찾는 다른 길**"이다.

실습 (3/3): 이 예가 보여주는 세 가지

1. **"연립방정식"의 실질.** "벨만방정식을 풀다" = n개 미지수 $V(s)$ 에 대한 n개 일차방정식. 3개면 손으로, 100개면 100×100 역산(귀찮지만 가능), 100만 개면 **사실상 불가능**.
2. **수렴은 "느리다"가 정석.** 오류가 sweep마다 대략 $\times\gamma$ 배. $\gamma = 0.9$ 이면 절반까지 ≈ 6.6회, 10^{-6} 까지 ≈ 120회. $\gamma = 0.99$ 라면 "수백~수천 sweep".
3. **수렴은 초기값에 무관.** 0으로 시작해도 $V = [100, -50, 10]$ 처럼 아무렇게나 잡아도 **같은** V^π 로 수렴(4.3절 바나흐 고정점 정리).

이 반복적 정책평가는 Sutton · Barto(2018) 표준 교과서 4장 "Dynamic Programming"과 동일. 4.3절 연습 2의 **비순환** MDP는 "터미널부터 거꾸로 대입"만으로 답이 나오는 특수 경우.

반복적 정책평가: 코드

상태가 많으면 다음을 값이 더 이상 안 바뀔 때까지 반복 근사:

```
def policy_evaluation(P, R, policy, gamma, theta=1e-6):
    n_states = len(P)
    V = [0.0] * n_states
    while True:
        delta = 0                       # 매 sweep 시작마다초기화 !
        for s in range(n_states):
            a = policy[s]
            v_new = R[s][a] + gamma * sum(prob * V[next_s]
                                          for prob, next_s in P[s][a])
            delta = max(delta, abs(v_new - V[s]))
            V[s] = v_new                # in-place: 갱신된값을즉시사용
        if delta < theta:
            break
    return V
```

$V(s)$**를 벨만 우변으로 갱신**

in-place(가우시) vs synchronous(야코비)

in-place (가우시) — 위 코드

- sweep **안에서도**, 앞쪽 상태가 갱신된 값을 뒤쪽이 **즉시** 사용.
- sweep 중반 V 는 "어떤" 가치함수의 우변도 아니지만, $\gamma < 1$ 이면 같은 고정점으로 수렴.

synchronous (야코비)

- sweep을 **완전히 끝낸 뒤**에야 갱신 (V 는 이전 sweep의 값).
- V_{new} 리스트를 따로 모았다가 교체.

실용적 차이

둘 다 **같은 답**에 수렴하지만 **속도**는 다를 수 있다. sweep당 비용은 동일하고, in-place가 갱신 정보가 sweep 안에 전파돼 보통 **2∼10배** sweep 수를 아낀다.

“정보의 전파”로 보는 수렴

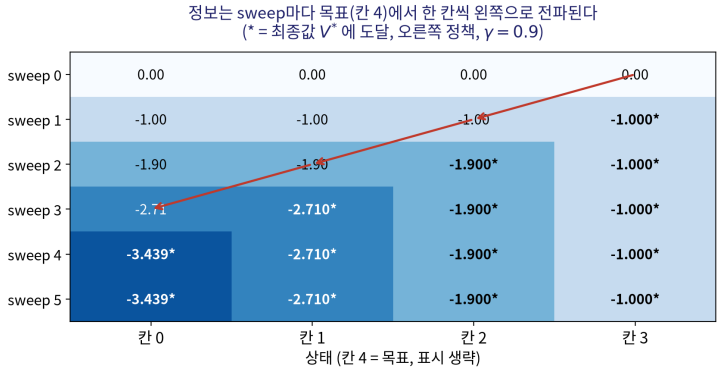
전파 직관으로 수렴 설명

경험칙

“정보는 한 sweep에 한 칸 전파된다.” $\Rightarrow$ sweep 수 $\approx$ 목표까지 최대 거리 $\times$ (할인이 충분히 작아질 때까지의 횟수). **순환** MDP에서는 “순환 한 바퀴 전파”.

In-place sweep 전파 예시

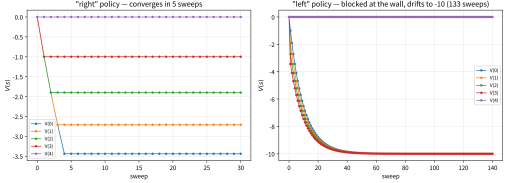
그림으로: sweep마다 한 칸씩 퍼지는 정보



히트맵의 각 행이 한 sweep. 목표(칸 4)에서 가까운 칸부터 차례로 V^* 값(빨간 글자 *)에 도달하며, 그 경계가 sweep마다 한 칸씩 왼쪽으로 넓어진다 — 위 “경험칙”의 그림 버전.

두 정책 나란히: “왼쪽”의 나쁨을 정량화

$$V^{\textbf{left}}(s) = -10$$



두 정책의 반복적 정책평가 수렴 곡선 ($V(0)$, in-place sweep 단위).

	V
right	$[-3.439, -2.710, -1.900, -1.000, 0]$
left	$[-10.000, -10.000, -10.000, -10.000, 0]$

- “오른쪽”은 **5 sweep** 만에 수렴 — 4.2절 “정책 반복이 첫 sweep에서 멈춘다”와 일치.
- “왼쪽”은 벽(칸 0)에 막혀 무한 루프 $\rightarrow$ **133 sweep**. 가치(-10)가 γ 로 할인돼 **유한**하게 수렴 — “$\gamma < 1$ 이면 무한 합 이 수렴”(3.2절)과 같은 원리.

행동가치 Q^π: "다음 행동을 포함해" 평가

V^π 를 안다 $\iff$ Q^π 를 안다

다리 $V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a)$ 로 정리하면, **어느 한쪽을 안다면 다른 쪽은 즉시 계산**.

"오른쪽" 정책에서 $V^{\text{right}} = [-3.439, -2.710, -1.900, -1.000, 0]$ 와 $Q(s,a) = R(s,a) + 0.9\, V(s')$ 로 직접 채운 Q^π 표:

s	$Q(s, \text{left})$	$Q(s, \text{right})$	행의 arg max
0	-4.095	-3.439	right
1	-4.095	-2.710	right
2	-3.439	-1.900	right
3	-2.710	-1.000	right
4	0.000 (동점)	0.000 (동점)	right (목표라 무의미)

Q^π 표로 정책 개선 확인

벨만 최적방정식: max 한 글자가 가르는 두 문제

강화학습이 정말로 풀고 싶은 문제는 **"가장 좋은 정책을 찾는 것"**. 최적 가치 $V^*(s)$ 는:

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right]$$

기대방정식과의 차이

형태는 거의 같지만, 특정 $\pi(s)$ 가 고른 행동 **하나** 대신 **모든 행동 중 최선**($\max_a$). 이 **한 글자**가 "이 정책이 얼마나 좋은가 평가"와 "가장 좋은 정책 찾기"라는 **두 문제를 가른다**.

왜 "정의"인가 — 원리 최적성

모든 정책($2^5 = 32$, $4^{100} \approx 10^{60}$ 개)을 직접 비교할 수는 없는데, 이 식은 "비교"를 **각 상태에서 한 스텝만**으로 압축. 근거: **원리 최적성**(최적 경로의 부분 경로도 최적) — s' 이후의 선택은 s 이전 선택과 **독립**(마르코프 성질)이라, "각 상태 한 스텝만 최적"을 재귀적으로 이으면 "전체적으로 최적". (이 성질이 없는 비마르코프 문제에서는 DP 자체가 적용 불가.)



Andrey Markov (1856–1922), 1886년 사진.
마르코프 성질(Markov property) — MDP의 “M”이 여기서 왔다.

Q^* 버전: DQN의 출발점

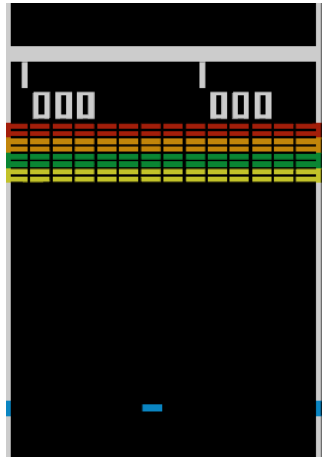
$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') = R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, \max_{a'} Q^*(s',a')$$

$V^* = \max_a Q^*$ 이므로 Q^* **만으로 닫혀**:

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

Q^* 를 알면 최적 정책이 직접 주어진다

이 관점이 **Week 9 (DQN, Mnih et al., 2015)**의 출발점 — DQN은 V^* 표가 아니라 Q^* 표(신경망)를 배우고, arg max 로 정책을 읽어낸다. 이번 절에서 Q^π 표를 직접 채워본 경험은, DQN의 "Q값을 추정하는 신경망"이 **무엇을** 추정하는지 이해하는 데 바로 쓰인다.



Atari Breakout 게임 화면.
DQN(Mnih et al., 2015, Nature)이 픽셀 입력만으로 학습한 49개 아타리 게임 중 하나 — Q^* 를 신경망으로 근사한다.

역사: “벨만”과 “동적계획법”의 이름

리처드 벨만 — 동적계획법 이름의 기원

- **큰 문제 = 작은 부분 문제 + 재귀**
- **동적 = 시간, 계획법 = 계획**



Richard Bellman (1920–1984) — RAND Corporation.
1957년 저서 Dynamic Programming에서 이름과 이론을 정립.

자주 하는 실수 (1/2)

- V^{π} 와 V^{*} 혼동 금지

자주 하는 실수 (2/2)

- $\gamma = 1$ **을 아무 고민 없이 쓴다.** 축소 사상 성질 $\|TV_1 - TV_2\| \leq \gamma \|V_1 - V_2\|$ 이 $\gamma = 1 < 1$ 이 **아니므로** 성립하지 않고 수렴이 보장되지 않는다(유한 에피소드만). 보상이 무한히 계속되는 문제에서 리턴 자체가 **발산**. “할인 없이 다 더하기”는 **보상 합이 유한**한 문제에만 안전.

실습: 5칸 GridWorld에서 두 정책 나란히

4.2절의 5칸 GridWorld(칸 0∼4, 칸 4가 목표, 걸음마다 -1, $\gamma = 0.9$)에 **두 정책**을 모두 평가 — 이 비교가 4.2절 정책개선을 **예고**:

```
n_states, n_actions, gamma = 5, 2, 0.9   # 0=left, 1=right
P, R = {}, {}
for s in range(n_states):
    P[s], R[s] = {}, {}
    for a in range(n_actions):
        if s == 4:
            P[s][a], R[s][a] = [(1.0, 4)], 0.0   # 목표 = 종료
        else:
            ns = max(0, s-1) if a == 0 else min(4, s+1)
            P[s][a], R[s][a] = [(1.0, ns)], -1.0

policy_right = [1, 1, 1, 1, 0]     # 모든칸에서오른쪽  ""
policy_left  = [0, 0, 0, 0, 1]     # 모든칸에서왼쪽  ""

V_right = policy_evaluation(P, R, policy_right, gamma)
V_left  = policy_evaluation(P, R, policy_left, gamma)
# V_right: [-3.439, -2.710, -1.900, -1.000, 0.000]
# V_left : [-10.000, -10.000, -10.000, -10.000, 0.000]
```

"오른쪽"이 **모든 칸에서** "왼쪽"보다 낫다($-3.439 > -10, \ldots$). 이 **상태별 비교**가 4.2절 정책개선의 본체 — "각 상태에서, **현재 정책의** V^π **기준으로** 더 나은 행동이 있으면 그 행동으로 바꾼다".

FAQ

- 정책평가만으로 최적성 판별 불가
- 가 1에 가까울수록 수렴 느려짐

4.1절 마무리: 세 개의 도구

4.1절에서 만든 것

1. **벨만 연산자** T^π — “가치함수를 먹여 우변을 돌려주는” 함수. 모든 반복법 = **이것을 같은 값에 계속 대입**.
2. V^π**/**Q^π **다리** $Q^\pi(s,\pi(s)) = V^\pi(s)$ — V/Q 방정식 사이를 오가는 두 줄짜리 다리.
3. **벨만 최적방정식** — 기대방정식의 $\pi(s)$ 을 $\max_a$ 로 바꾼 것. **최적 정책**을 정의하는 방정식, 원리 최적성이 “정의”임을 담보.

예습 확인 문제

4.2 정책 반복과 GridWorld 실습

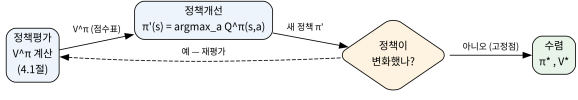
정책 반복: 두 단계가 서로를 먹여 살린다

1. **정책평가**: 현재 π 의 V^{π} 를 계산(4.1절).
2. **정책개선**: 각 상태에서 V^{π} 기준으로 더 나은 행동이 있으면 바꾼다:

$$\pi'(s) = \arg\max_{a} \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^{\pi}(s') \right]$$

멈춤 신호

정책이 **더 이상 안 바뀔 때까지** 반복하면 최적 정책에 도달. “개선이 아무것도 바꾸지 않았다”는 순간이 멈춤 신호.



평가와 개선이 서로의 출력을 입력으로 먹는다

왜 이 두 단계인가 + 개선 = Q^π 표 행별 argmax

평가 alone은 “지금 정책이 **얼마나 좋은가**” 만 답하고, “어떻게 **고쳐야** 하는가”를 말해주지 않음(4.1절 FAQ: 정책평가만으론 “최적인가” **판별 불가**).

개선 alone은 “더 나은 행동”을 V^π 기준으로 판단해야 하는데, **정책이 바뀌면 V^π 자체가 바뀌어** 다시 계산 필요 $\rightarrow$ 한 발도 못 뺌.

핵심 연결

개선 식의 우변 $R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s')$ 는 4.1절 $Q^\pi(s,a)$ **그 자체**. 즉 4.1절이 직접 채운 Q^π **표가 정책개선의 점수표**이고, 개선은 **각 행에서 최댓값을 따는 동작**. “오른쪽” 정책은 자기 Q^π 기준으로 **이미 바꿀 것이 없는** 상태 (정책개선의 **고정점**).

이론 (1/3): 질문 1 — “안 나빠지는가”?

개선한 정책이 정말로 안 나빠지는가?

정책개선 정리 (단조 증가)

π' 가 π 를 **모든 상태에서 동시에** 개선한 정책이면, 모든 s 에서 $V^{\pi'}(s) \geq V^{\pi}(s)$.

$$V^{\pi'}(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^{\pi'}(s') \right] \geq R(s,\pi'(s)) + \gamma \sum_{s'} P(s'|s,\pi'(s))\, V^{\pi'}(s')$$

이론 (2/3): 증명은 두 줄

$$R(s,\pi'(s))+\gamma\sum_{s'}P(s'|s,\pi'(s))\,V^{\pi}(s') \geq R(s,\pi(s))+\gamma\sum_{s'}P(s'|s,\pi(s))\,V^{\pi}(s') = V^{\pi}(s)$$

두 줄 합치기:

$$V^{\pi'}(s) \;\geq\; R(s,\pi'(s)) + \gamma\sum_{s'}P(s'|s,\pi'(s))\,V^{\pi}(s') \;\geq\; V^{\pi}(s) \qquad \square$$

한 칸 손계산으로 체감

이론 (3/3): 멈춤 = 최적, 그리고 유한성

질문 2. 멈춤 지점이 "최적"인가? 멈춤 조건: **모든** s **에서** $\pi(s) \in \arg\max_a Q^\pi(s,a)$ (자기 기준으로도 바꿀 것이 없음). 이 조건에서 **직접** 읽히는 것:

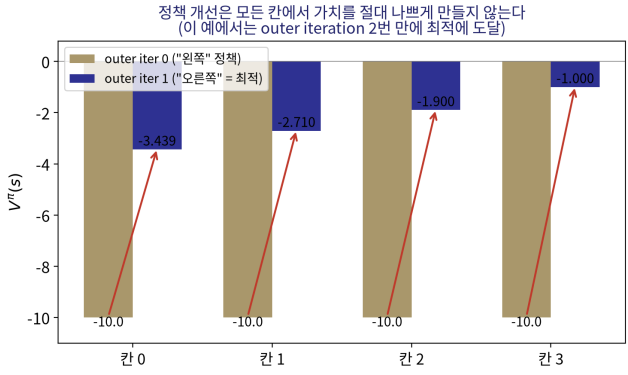
$$V^\pi(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\pi(s') \right] \quad \text{(모든 } s\text{)}$$

고정점에서 $V^\pi = V^*$

유한성

결정론적 정책 총수 m^n(유한) + 가치 감소하지 않음(위 정리) + **동점 처리 고정**(항상 최소 행동 번호) $\rightarrow$ 정책 수열은 유한집합 위 **결정적 함수의 궤적** $\rightarrow$ 언젠가 멈추거나 **주기 궤도** 진입. 주기 궤도 안의 모든 정책은 **같은 가치**를 갖고, 그 값은 벨만 최적방정식을 만족 $\rightarrow$ **어느 경우든 가치 기준의 최적**. (고전적 결과: 바깥 반복 $\leq n(m-1)$ 회.)

그림으로: 정책 개선의 단조성



이 장의 예제에서 초기("왼쪽") 정책의 V(황토색)가 개선 1회만으로 모든 칸에서 최적값(파랑, V^*)으로 증가한다 — "개선은 절대 나빠지지 않는다"는 정리의 구체적 모습.

실전적 상한 + “평가만 하면 안 된다”는 반례

단조 증가는 “나빠지지 않는다”지만 “얼마나 빨리 좋아지는가” 는 주지 않음. 수축 성질(4.3절)로부터:

$$\max_s \left|V^{\pi'}(s) - V^{\pi}(s)\right| \le \frac{1-\gamma}{\gamma} \max_s \left|V^{*}(s) - V^{\pi}(s)\right|$$

실무적 메시지

초기 정책이 아무리 엉망이어도, γ 가 1에서 충분히 멀지만 한 한 바퀴(평가+개선)에 오차가 고정 비율(γ 배)로 줄어든다. $\gamma = 0.99$ 처럼 1에 가까우면 상한도 느려져 전체 비용이 크게 불어남.

“평가만 하면 안 된다”는 반례

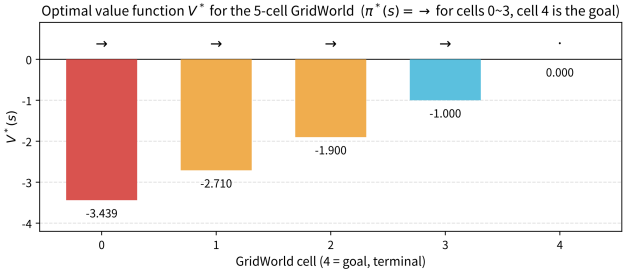
“왼쪽” 정책($V = -10$)을 **평가만** 하면 “가치 -10 이다”가 **전부**. 개선 단계를 **한 번만** 끼워도 **칸 3이 right로** 바뀌고($V(3) : -10 \to -1$), 그 정보가 다음 평가에서 **칸 2로, 다음에 칸 1로** 퍼짐. **평가 alone**은 점수만 찍고, **개선 단계**가 점수표를 “갈아탈 방향”으로 번역 — 두 단계의 **곱**(점수 측정 × 방향 추출)이 정책 반복의 힘.

실습: 5칸 GridWorld의 policy_iteration

```
def policy_iteration(P, R, n_actions, gamma):
    policy = [1] * n_states   # 처음엔무조건오른쪽
    while True:
        V = policy_evaluation(P, R, policy, gamma)   # 1. 평가
        stable = True
        for s in range(n_states):                    # 2. 개선
            best_a = max(range(n_actions),
                         key=lambda a: R[s][a]
                             + gamma * sum(p*V[ns] for p, ns in P[s][a]))
            if best_a != policy[s]:
                policy[s] = best_a
                stable = False
        if stable:                    # 멈춤: 바뀐것이없으면
            return V, policy
```

실행 결과: $V = [-3.439, -2.710, -1.900, -1.000, 0.000]$, policy $= [1, 1, 1, 1, 0]$, **outer 2회**, sweep 총합 **10**. 시작 정책이 이미 "무조건 오른쪽"이라 **첫 정책평가에서 곧바로 최적**이 나왔고, 개선 단계에서 "더 바꿀 게 없다"는 것만 한 번 확인. $V(0) = -3.439$ 는 "칸 0→4까지 4번 이동의 할인 비용" $(-1 - 0.9 - 0.81 - 0.729 \approx -3.439)$ 과 정확히 일치.

5칸 GridWorld의 V^* 와 π^* — 정답 기준



5칸 GridWorld의 최적 가치함수 V^* 와 최적 정책 π^*.

- 칸 4(목표, 종료)에서 칸 0까지 각 칸의 V^* 는 “**목표까지 걸음 수의 할인 비용**”: $-1, -1-0.9,$ $-1-0.9-0.81, \ldots$
- 최적 정책: **모든 칸에서 “오른쪽”**.
- 이 $V^* = [-3.439, -2.710, -1.900, -1.000, 0]$ 가 **이번 장 모든 알고리즘**(정책 반복 · 수정된 정책 반복 · 가치 반복)의 **정답 기준**.

최악의 출발: "왼쪽"에서 시작하면 한 칸씩 전파

outer	정책	sweep	V
1	$[L, L, L, L, \cdot] \to [L, L, L, R, \cdot]$	133	$[-10, -10, -10, -10, 0]$
2	$[L, L, L, R, \cdot] \to [L, L, R, R, \cdot]$	133	$[-10, -10, -10, -1, 0]$
3	$[L, L, R, R, \cdot] \to [L, R, R, R, \cdot]$	133	$[-10, -10, -1.9, -1, 0]$
4	$[L, R, R, R, \cdot] \to [R, R, R, R, \cdot]$	133	$[-10, -2.71, -1.9, -1, 0]$
5	$[R, R, R, R, \cdot] \to$ (변화 없음)	5	$[-3.44, -2.71, -1.9, -1, 0]$

(L = left, R = right, $\cdot$ = 목표. **총** $133 \times 4 + 5 = 537$ **sweep**.)

왜 **칸 3**이 가장 먼저 바뀌는가?

outer 1에서 V^{left} 가 모두 -10 인 상태에서 $Q(3, \text{left}) = -10$ 인데
$Q(3, \text{right}) = -1 + 0.9 \times 0 = -1$ — **목표($V = 0$)가 바로 옆**인 칸 3에서만 right가 압도적으로 낫다. 칸 0~2는 $Q(s, \text{right}) = -10$ 으로 left와 **동점** $\to$ 'max'가 기존 left를 **그대로 둔다**.

정보 전파(바깥 반복 버전) + 세 방법 나란히

같은 알고리즘, 시작 정책만 바꿔도

"오른쪽" 출발: 총 sweep **10** vs "왼쪽" 출발: 총 sweep **537** — **약 54배** 차이. "정책 반복 속도는 초기 정책에 **민감하다**"의 실질, 그리고 4.3절 가치 반복이 "초기 정책에 **무관하게** V^* 로 직접 간다"는 설계 의도의 동기.

"왼쪽" 출발, $\theta = 10^{-6}$ 기준 세 방법:

방법	바깥 반복	내부 sweep	**총 sweep**
정책 반복 (완전 수렴)	5	$133 \times 4 + 5$	**537**
수정된 정책 반복 ($k = 2$)	5	2×5	**10**
수정된 정책 반복 ($k = 1$) = 가치 반복	5	1×5	**5**

수정된 정책 반복: k sweep만 평가 후 개선

```python
def modified_policy_iteration(P, R, n_actions, gamma, k):
    """k = 내부정책평가  sweep 수. k이면=1 가치반복과동일  ."""
    policy = [0] * n_states   # 왼쪽에서" 시작
    V = [0.0] * n_states
    while True:
        for _ in range(k):                    # 평가: k 만sweep
            for s in range(n_states):
                a = policy[s]
                V[s] = R[s][a] + gamma * sum(p * V[ns] for p, ns in P[s][a])
        stable = True
        for s in range(n_states):             # 개선: argmax
            best_a = max(range(n_actions),
                         key=lambda a: R[s][a] + gamma * sum(p * V[ns] for p, ns in P[s][a]))
            if best_a != policy[s]:
                policy[s] = best_a
                stable = False
        if stable:
            return V, policy
```

실행(모두 "왼쪽" 출발, 정책 변화 여부로 멈춤): $k = 1, 2, 3, 5$ 모두 **바깥 반복 5**, 최종 $V = [-3.439, -2.710, -1.900, -1.000, 0]$ **동일** (총 sweep = $5k$).

$k = 1$ 은 가치 반복: “같은 연산의 다른 분해”

$k = 1$ 확인

“평가 1 sweep” = 각 상태의 V 를 $\max_a$[보상 + 할인 미래값]으로 갱신하는 한 바퀴(정책을 “평가 중”에 argmax로 교체), “개선”은 이미 그 sweep 안의 argmax로 **흡수**. 두 표현(평가+개선 분리 vs 한 sweep에 통합)은 **동일한 연산의 다른 분해**.

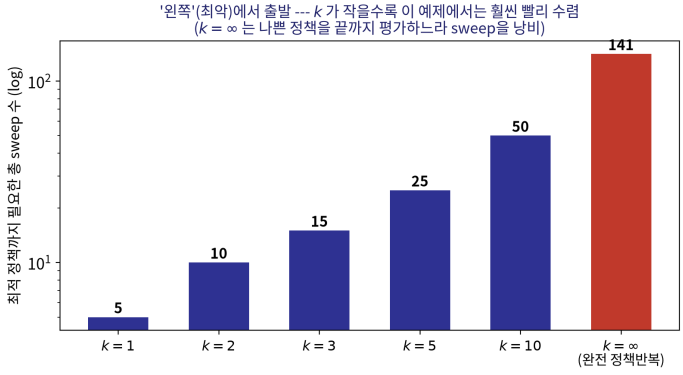
k 를 1에서 ∞(완전 수렴)로 늘리면 가치 반복에서 정책 반복으로 **연속적으로** 이어진다:

$$\text{가치 반복}(k = 1) \to k = 2 \to k = 3 \to \cdots \to \text{정책 반복}(k = \infty)$$

k 가 작아도 왜 최적인가

k sweep의 정책평가 연산 $T^{\pi,k}$ 는 “T^{π} 를 k 번 적용”이므로 여전히 γ^k**-축소 사상**. 바나흐 정리(4.3절)와 “고정점에서 멈추면 최적” 논증은 **변함 없이 적용** — **수렴 속도만** k 에 따라 변한다.

그림으로: k 를 얼마로 잡을 것인가



"왼쪽"(최악)에서 출발해 수렴까지 필요한 총 sweep 수(로그축, 직접 계산). k 가 작을수록(가치 반복에 가까울수록) 이 예제에서는 훨씬 빨리 수렴하고, $k = \infty$(완전 정책반복)는 나쁜 정책을 끝까지 평가하느라 sweep을 낭비한다.

자주 하는 실수

- **"평가 $\rightarrow$ 개선" 순서를 뒤집거나, 개선 전에 평가를 안 한다.** 개선 식의 V^{π} 는 **현재** 정책 가치. **이전** V 로 개선하면 "낡은 점수표로 갈아탈 방향을 고르는" 셈 $\rightarrow$ 단조 증가 전제가 **깨짐**. `V = policy_evaluation(...)`을 `while` **안쪽**에 두는 것이 맞고, 루프 밖에서 한 번만 계산해 재사용하면 안 된다.

실습: 동점 MDP — 정책은 다르고, V^* 는 유일

"동점이 있으면 결과 정책은 달라져도 **가치는 항상 같다**"를 작은 MDP로 확인. 상태 0이 **두 개의 문**(행동 0, 1)을 갖고 각각 방 1 / 방 2(각각 $+1$ 주고 **self-loop 종료**)로 가는 3상태 MDP:

```
# 상태 0: 두개의문  . 방 1,2: self-loop 종료()
P[0][0], R[0][0] = [(1.0, 1)], 1.0    # 문 a0 -> 방 1 (+1, 종료)
P[0][1], R[0][1] = [(1.0, 2)], 1.0    # 문 a1 -> 방 2 (+1, 종료)
P[1][0], R[1][0] = [(1.0, 1)], 0.0    # 방 1: self-loop
P[2][1], R[2][1] = [(1.0, 2)], 0.0    # 방 2: self-loop
# 나머지( P, 은R 0. 방 1는·2 어떤행동을해도자기자신    )

def policy_iteration_tie(P, R, n_actions, gamma, tie_break):
    policy = [0] * n_states
    while True:
        V = policy_evaluation(P, R, policy, gamma)
        stable = True
        for s in range(n_states):
            qs = [R[s][a] + gamma * sum(p*V[ns] for p, ns in P[s][a])
                  for a in range(n_actions)]
            best = max(qs)
            cands = [a for a in range(n_actions) if abs(qs[a] - best) < 1e-12]
            chosen = cands[0] if tie_break == 'first' else cands[-1]
            if chosen != policy[s]:
                policy[s] = chosen
                stable = False
        if stable:
            return V, policy
```

동점 실행 결과: 정책 2가지, V^* 1가지

동점 처리	결과 policy	V
'first' (최소 행동 번호)	$[0, 0, 0]$	$[1.0000, 0.0000, 0.0000]$
'last' (최대 행동 번호)	$[1, 1, 1]$	$[1.0000, 0.0000, 0.0000]$

- 상태 0에서 두 행동: $Q(0, \text{left}) = 1 + 0 = 1$, $Q(0, \text{right}) = 1 + 0 = 1$ — **완벽한 동점**.
- **결과 정책은 완전히** 다르고($[0, 0, 0]$ vs $[1, 1, 1]$), V 는 **완전히** 같다($[1, 0, 0]$).

유한성 논증이 약속한 그대로

동점 MDP에서 **"최적 정책"은 유일하지 않을 수 있다**(여기서 2개 이상 공존), 하지만 **"최적 가치 V^*"는 유일**. 실전에서 동점이 중요한 이유: (a) 같은 V^* 를 전제로 정책은 **추가 기준**(더 안전한 행동 선호, 물리적 비용)으로 골라야 하며, (b) "정책이 다르다"는 이유로 "알고리즘 실패"로 판단하면 **안 된다** — V^* **가 일치하면 성공**.

FAQ

- **정책평가를 항상 끝까지 해야 하나요?**

4.2절 마무리

4.2절에서 만든 것

1. **정책 반복** = 평가 $\rightarrow$ 개선을 번갈아, “개선이 아무것도 안 바꿀 때” 멈춤.
2. **정책개선 정리(단조 증가)** + **유한성** $\rightarrow$ “멈추면 최적”(멈춤 = 벨만 최적방정식 성립, 해 유일 $\Rightarrow V^{\pi} = V^{*}$).
3. **수정된 정책 반복**(평가 k sweep + 개선) — $k = 1$ 은 **가치 반복**, $k = \infty$ 는 **정책 반복**. “같은 알고리즘의 k 스펙트럼”.

예습 확인 문제

4.3 가치 반복, 수렴 증명, 그리고 모델 기반이라는 전제

가치 반복: “한 줄 차이”가 최적을 찾는다

가치 반복: 벨만 최적방정식 직접 풀기

```python
def value_iteration(P, R, n_actions, gamma, theta=1e-6):
    n_states = len(P)
    V = [0.0] * n_states
    while True:
        delta = 0
        for s in range(n_states):
            v_new = max(R[s][a] + gamma * sum(prob * V[next_s]
                        for prob, next_s in P[s][a])
                        for a in range(n_actions))   # <- max_a
            delta = max(delta, abs(v_new - V[s]))
            V[s] = v_new
        if delta < theta:
            break
    policy = [max(range(n_actions),
                  key=lambda a: R[s][a] + gamma * sum(prob * V[next_s]
                                for prob, next_s in P[s][a]))
              for s in range(n_states)]
    return V, policy
```

“한 줄”의 차이: $a = \pi(s) \to \max_a$

- **정책평가** = **정책 연산자** T^π 를 반복하는 것: $V \xrightarrow{T^\pi} \cdots$
- **가치 반복** = **최적 연산자** $T^* V(s) = \max_a [R + \gamma \sum P\, V]$ 를 반복하는 것: $V \xrightarrow{T^*} V_1 \xrightarrow{T^*} V_2 \to V^*$.

$T^{\pi} \to T^{*}$: 기대 열 vs 최적 열

정책평가와 가치 반복의 연산자 차이

$$T^{*}V(s) = \max_{a}\left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\,V(s')\right] \qquad (V \xrightarrow{T^{*}} V_1 \xrightarrow{T^{*}} V_2 \to V^{*})$$

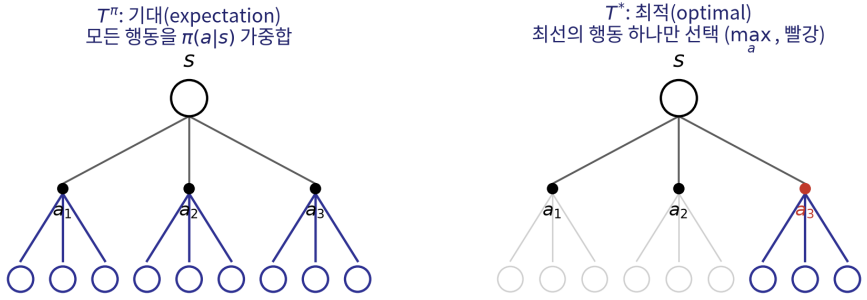
T^{*} 를 k 번 적용 = "k스텝만 내다보기"

5칸 GridWorld, $V_0 = [0,0,0,0,0]$ 로 시작해 T^{*} 를 **한 번씩만**(synchronous) 적용:

k	$V_k(0)$	$V_k(1)$	$V_k(2)$	$V_k(3)$
0	0.000	0.000	0.000	0.000
1	−1.000	−1.000	−1.000	−1.000
2	−1.900	−1.900	−1.900	−1.000
3	−2.710	−2.710	−1.900	−1.000
4	−3.439	−2.710	−1.900	−1.000
5	−3.439	−2.710	−1.900	−1.000

그림으로: backup diagram으로 보는 T^{π} vs T^{*}

Backup diagram: 기대 열 vs 최적 열의 차이



위 원 = 상태 s, 검은 점 = 행동, 아래 원 = 다음 상태. 왼쪽(T^{π})은 모든 행동을 $\pi(a|s)$ 로 가중합, 오른쪽(T^{*})은 가장 좋은 행동 하나(빨강)만 골라 그 가지를 따라간다.

지평선이 한 칸씩 넓어지고, $k = 4$ 에서 고정

$V_k(s)$ 해석: 정확히 k스텝 최적 행동

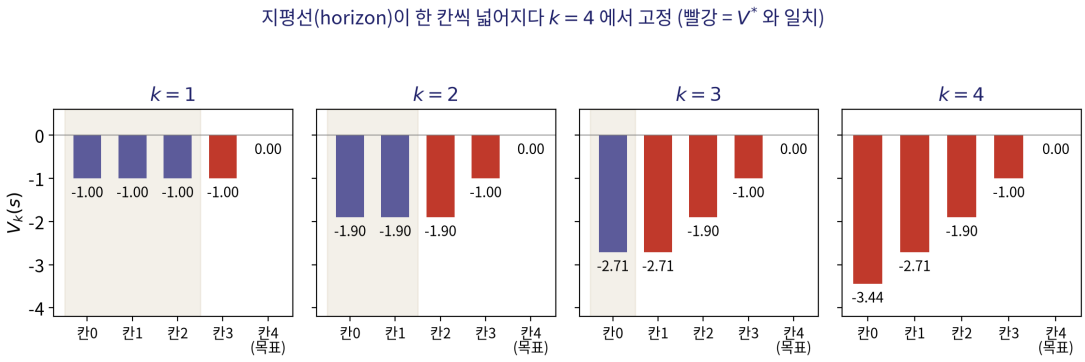
지평선(horizon)이 매 스텝 한 칸씩 오른쪽으로 넓어진다

4.1절 “정보는 한 sweep에 한 칸 전파”와 **정확히 같은 현상**. $k = 4$ 가 되면 칸 0에서도 목표까지 거리(4칸)를 다 보게 되어 **고정**(-3.439). $k = 4$ **에서 더 이상 안 변한다** — 목표까지 **최장 거리 4**이므로 “4스텝만”과 “무한히” 보는 것이 **같은 답**.

이 직관의 두 의미

(1) k **가 충분히 크면** V_k **가** V^* **에 수렴** — “왜”가 아니라 **“몇 스텝이면”의 눈금자**(목표까지 최대 거리 + γ^k 가 충분히 작아질 때까지의 k). (2) “내다보는 스텝 수를 늘려가며 근사”는 Week 14 **MPC** (Mayne et al., 2000)과 **동일 사상** — MPC는 매 결정 시점 “앞으로 k스텝만”을 고르고 k 키울수록 장기 계획 반영, 가치 반복은 k 를 **스스로 키워** $k = \infty$(= V^*)까지 닫아가는 절차.

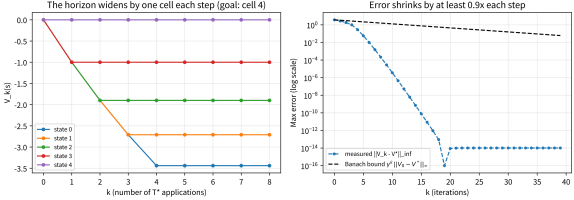
그림으로: 지평선이 넓어지는 4장의 스냅샷



$k = 1, 2, 3, 4$ 각각에서 칸별 $V_k(s)$(막대)와 아직 목표에 닿지 못한 칸(연한 배경). $k = 4$ 에서 모든 칸이 V^*(빨간 막대)에 도달해 더 이상 바뀌지 않는다.

실습: 정책반복 vs 가치반복, 같은 고정점

```
V_vi, policy_vi = value_iteration(P, R, n_actions, gamma)
# V      = [-3.439, -2.710, -1.900, -1.000, 0.000]  # 정책반복과동일
# policy = [1, 1, 1, 1, 0]                          # 정책반복과동일
# 반복횟수 : 5
```



(좌) 지평선을 한 칸씩 넓혀가는 과정, (우) 오차 로그 곡선 — 실측 오차(파란 점선)가 바나흐 상한 γ^k(검정) **아래**를 따라감.

- 두 알고리즘 모두 **정확히 같은** V^* 와 최적 정책 — **같은 고정점**(벨만 최적방정식) 을 찾아가기 때문(해 유일, 아래 증명).
- "수렴까지 **계산의 성격**"이 다름: 정책반복은 바깥 반복 2번(내부 평가 sweep 총 10) vs 가치반복은 단일 루프 5번.
- **일반적으로**: 정책반복은 바깥 반복은 적지만 각 반복(평가)이 **무거울 수** 있고, 가치반복은 **가볍지만** 반복 횟수가 더 필요할 수 있음. 어느 쪽이 총 계산량이 적은지는 **문제마다 다름**.

in-place vs synchronous: 여기서 왜 일치하나?

위 `value_iteration`은 **in-place**(sweep **중**에도 갱신된 값을 사용)인데, 이 5칸 예에서는 synchronous와 **매 스텝 값이 정확히 일치** — 위 표의 V_k 행이 바로 **두 버전 공통의 추적궤적**.

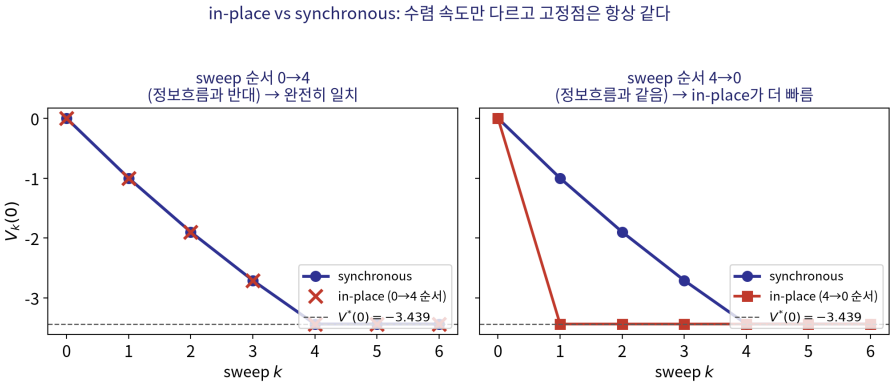
이유: 최적 전이가 오른쪽 방향뿐

이 GridWorld의 **최적 전이가 오른쪽 방향만**이므로, sweep을 **0→4 순서**로 돌리면 갱신 정보가 "아직 안 갱신된 → 이미 갱신된 상태로만" 흘러간다(sweep 순서가 정보 흐름과 **반대** 방향).

sweep 순서 = 정보 흐름 방향이면

sweep 순서가 정보 흐름과 **같은** 방향(예: 상태 순서를 4→0 으로 뒤집으면)이면 in-place가 **한 sweep 안에 정보를 더 멀리 전파해 더 빠르게 수렴**할 수 있다 (4.1절). **같은 고정점**으로 수렴한다는 보장은 **변하지 않음** — 차이는 **속도**뿐.

그림으로: 두 sweep 순서, 두 개의 결과



칸 0의 $V_k(0)$ 궤적. 왼쪽(sweep 순서 $0 \to 4$)은 in-place와 synchronous가 완전히 겹친다 — 지금 예제가 이 경우. 오른쪽(가상의 $4 \to 0$ 순서)은 in-place가 한 sweep 만에 수렴 — 정보 흐름과 sweep 순서가 같을 때의 이득.

정책반복과 가치반복: “같은 알고리즘의 두 극한”

수정된 정책 반복을 k 에 따라 나열

k	이름	바깥 반복	총 sweep	결과 $V(0)$
1	**가치 반복**	5	5	-3.439
2	수정된 정책반복	4	8	-3.439
4	수정된 정책반복	2	8	-3.439
∞	**정책 반복**	2	10	-3.439

“다른 알고리즘”이 아니라 연속 스펙트럼

가치반복과 정책반복은 따로따로 배운 “다른 알고리즘”이 아니라, **같은 프레임워크의 연속 스펙트럼 양 끝단**이다. k 를 키우면 **바깥 반복은 줄고** 각 반복이 **무거워지며**, 극단 $k = 1$ 이 **가치반복**, $k = \infty$ 가 **정책 반복**.

k 가 적어도 최종 답은 그대로

- $k = 1$ 부터 $k = \infty$ 까지 **모두 정확히 같은** V^* 에 도달 — 차이점은 오직 **바깥 반복 횟수와 총 sweep 수**(= 총 계산량)뿐.

실전 표준 트릭

"정확한 답"은 변하지 않으니 **총 계산량만** 줄이려 k 를 작게(2∼10 정도) 잡는다. k 가 **너무 작으면** 개선의 방향 감각이 떨어져 바깥 반복이 늘고, **너무 크면** 각 반복이 비싸지는 — 이 **trade-off**가 k 라는 하나의 수로 조절되는 구조.

왜 이 반복이 수렴하는가: γ-축소 사상

T는 축소 사상(contraction mapping)이다

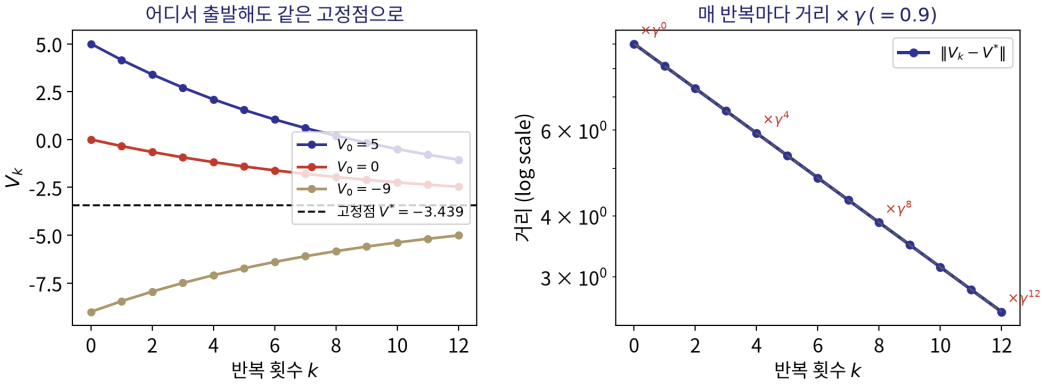
$$\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

의미

T 를 **한 번 적용할 때마다** 두 함수의 (최대) 차이가 **최소 γ 배로 줄어든다**.

그림으로: γ-축소가 뜻하는 것

T 는 γ-축소 사상: 반복마다 오차가 최소 γ 배 줄어든다



왼쪽: 시작값이 달라도($V_0 = 5, 0, -9$) 모두 같은 고정점 V^*로 모인다. 오른쪽: 오차 $\|V_k - V^*\|$ 가 매 반복 최소 γ 배(로그축에서 직선)로 줄어든다.

바나흐 고정점 정리

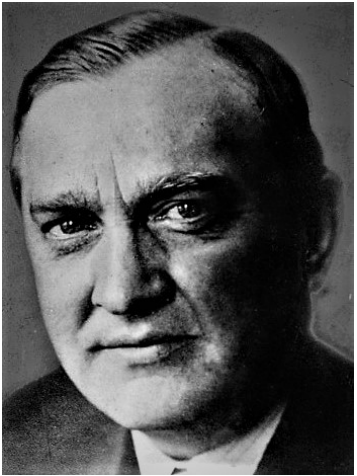
바나흐 고정점 정리 (Banach fixed point theorem)

축소 사상은 항상 **유일한 고정점**을 가지며, **어디서 출발해** 반복해도 **그 고정점에 수렴**한다.

벨만 연산자에 적용하면:

- 벨만 최적방정식을 만족하는 V^* 가 **존재하고 유일**하다.
- **가치 반복**(임의의 값에서 시작해 T 를 반복)이 실제로 **그 값에 도달**한다.
- V^* 를 알면, 각 상태에서 그 max 를 달성하는 **결정론적 정책** π^* 를 만들 수 있다 — 행동 집합이 **유한**하므로 유한집합의 최댓값은 **항상 어딘가에서 달성**된다.

4.2절의 "멈춤 = 최적" 논증이 여기서 완결: 멈춤 조건이 벨만 최적방정식이고, 그 해가 **유일**하니 멈춘 정책의 가치가 바로 V^* 다.



Stefan Banach (1892–1945).
1920년대 바나흐 고정점 정리(Banach fixed point theorem)를 증명.

증명은 “존재”이지 “계산 가능성”이 아니다

중요한 구별

이 증명이 보장하는 건 **“존재”**이지 **“계산 가능성”이나 “저장 가능성”**이 아니다.

- **바둑**처럼 상태 수가 **초천문학적으로** 많으면 (약 10^{170}), π^* 가 **이론적으로 존재해도** 모든 상태에 대해 **표로 저장하는 것 자체가 불가능**. **표 대신 신경망으로 근사**하는 방법이 필요 — **Week 9 DQN**(Mnih et al., 2015).

같은 수학의 또 다른 얼굴

“반복 대입이 수렴한다”는 증명은, 그래프 위에서 **무작위 걷기의 정상분포**를 구하는 **PageRank**(Brin & Page, 1998) 알고리즘과 **정확히 같은 수학**(축소 사상의 반복 적용). (ML1 Week 13.3에서 이미 만났다.)

모델 기반이라는 전제: P 를 "정확히" 안다

이번 장의 **모든 알고리즘**은 전이확률 $P(s'|s,a)$ 를 **정확히 알고 있다는** 전제 위에 서 있다.

이 전제가 뜻하는 것

체스라면 **"이 수를 두면 상대는 어떻게 반응할 확률이 얼마인가"를 미리 다 알아야** 한다는 뜻.
현실에서는 대부분 이걸 알 수 없다.

이 장을 받치는 뼈대

최적해 = 벨만방정식의 고정점. 모델 기반이라는 **제약이 있지만**, 이후 모든 장의 **기준점**: Week 6 TD = 이 반복을 "모델 대신 **경험으로**" 근사한 것, Week 9 DQN = 같은 벨만 최적방정식의 고정점을 **신경망**으로 찾아가는 것.



이세돌 9단, 2016년.
같은 해 AlphaGo와의 대국 — 바둑은 규칙(P)이 완전히 알려진 드문 사례라 대량의 시뮬레이션(계획)이 가능했다.

동적계획법의 핵심 아이디어

“큰 문제 = 작은 부분 문제 + 재귀”

이 패턴은 **컴퓨터공학 전반**에서 반복해서 등장한다. 강화학습은 그 패턴을 “미래의 **불확실한** 보상을 어떻게 **지금** 계산할 것인가”라는 문제에 적용한 것.

- 재귀 구조(벨만방정식)로 **무한한 미래**를 **한 스텝**으로 압축.
- 그 반복이 **항상 정답에 수렴**한다는 것 자체가, **축소 사상(contraction mapping)**이라는 **하나의 수학적 성질**로 **보장**된다.

연습문제 (핵심 확인)

1. **(코딩)** 3-state MDP에서 `policy_evaluation`과 `value_iteration` (핵심 줄이 빈칸)을 완성 — **차이점 한 줄**: `policy_evaluation`은 `a = policy[s]`를 따르고, `value_iteration`은 `max_a`를 취하는 것뿐.
2. **backward induction으로 V(0) = -1.9**
3. **축소 사상 수렴: 어떤 V_0 로도 V^* 도달**

FAQ: 실전에서 쓰나요?

- **조건이 맞으면 쓰인다**
  - P 를 **모른다**(model-free) $\rightarrow$ Week 5–7(경험만으로 학습).
  - 상태 공간이 **너무 커서** 표로 나타낼 수 없다 $\rightarrow$ Week 9 DQN(신경망으로 근사).

4.3절 마무리

4.3절에서 만든 것

1. **가치 반복** = `policy_evaluation`의 "한 줄"($a = \pi(s) \to \max_a$) — 최적 연산자 T^* 를 반복 대입.
2. T^* **를** k **번 =** k**스텝만 내다보기** — 지평선이 한 칸씩 넓어지고, k 가 목표까지 최대 거리에 달하면 고정.
3. **바나흐 고정점 정리** — T 가 γ-축소 사상 $\Rightarrow$ 고정점 **유일** + 어디서든 **수렴** $\Rightarrow$ "멈추면 최적" 완결.
4. **모델 기반 전제**(P 를 정확히) — 이 장의 모든 알고리즘이 서 있는 땅, 그리고 Week 5–6 (model-free) · Week 9 (근사)가 채울 **빈자리**.

4주차 정리

1. **4.1 벨만방정식을 "풀다"는 = 반복 대입** — 벨만 연산자 T^π 를 같은 값에 대입하면(연립방정식 직접 풀이와 **같은 고정점**을 찾되, n 이 커지면 **반복만이 표준**) 초기값에 무관하게 V^π 로 수렴. V^π/Q^π 다리, 그리고 max 한 글자로 가르는 **벨만 최적방정식**(원리 최적성).
2. **4.2 정책 반복: "멈추면 최적"** — 평가$\rightarrow$개선의 두 단계. **단조 증가**(증명은 두 줄) + **유한성** $\Rightarrow$ "개선의 고정점 = 벨만 최적방정식 성립 = V^* ". k 스펙트럼: $k=1$ 가치 반복 $\cdots$ $k=\infty$ 정책 반복.
3. **4.3 가치 반복과 바나흐** — T^* 반복은 "k스텝만 내다보기"를 $k=\infty$ 까지. **축소 사상**(바나흐 고정점 정리)이 존재·유일성· 수렴을 하나의 성질로 **보장**. **"모델 기반" 전제**의 한계 = Week 5–6·9의 동기.

다음 주 — Week 5. 몬테카를로 방법

"P 를 정확히 안다"는 전제를 **버린다**. 에피소드를 실제로 플레이하며 **경험만으로** V^π, Q^π 를 추정하는 방법 — 그리고 이 장이 세운 **정답 기준** (벨만 고정점, V^* 표)과 비교하며 model-free 알고리즘의 "정확성"을 평가하는 법.